Why to Use enumerate

Get the position and the item together, without a manual counter

The old way: manual counter

Both versions work, but they are noisy and easy to break (forgotten `i += 1`, off-by-one slips).

```
names = ["Asha", "Ravi", "Meera"]
i = 0
for name in names:
    print(i, name)
    i += 1              # easy to forget
for i in range(len(names)):
    print(i, names[i])   # index juggling
```

What enumerate really returns

A lazy iterator of `(index, item)` tuples. Wrap it in `list()` to look inside.

```
list(enumerate("hi"))  # → [(0, 'h'), (1, 'i')]
next(enumerate("hi"))  # → (0, 'h')
```

Why it wins

Benefit	Detail
Readable	intent is obvious: each item with its position
Safer	no `i += 1` to forget, no off-by-one
Flexible	works on lists, tuples, strings, files, generators, dict views
Lazy	produces one pair at a time; no extra list in memory

First, last, every other

```
for i, name in enumerate(names):
    if i == 0:
        print("first:", name)
    elif i % 2 == 0:
        print("even slot:", name)
```

OUTPUT

```
first: Asha
even slot: Meera
```

The fix: enumerate

```
for i, name in enumerate(names):
    print(i, name)
```

OUTPUT

```
0 Asha
1 Ravi
2 Meera
```

Start counting from 1

```
for n, name in enumerate(names, start=1):
    print(f"{n}. {name}")
```

OUTPUT

```
1. Asha
2. Ravi
3. Meera
```

WATCH OUT `start` changes the counter, **not** the list: with `start=1`, `names[n]` is the next item. Use `name` or `names[n - 1]`.

Find positions

```
nums = [5, 12, 7, 20]
[i for i, n in enumerate(nums) if n > 10]  # → [1, 3]
```

Change items in place

Assigning to the loop variable changes nothing; assign through the index.

```
for i, n in enumerate(nums):
    nums[i] = n * 2
print(nums)              # → [10, 24, 14, 40]
```

With zip and dict.items()

Put parentheses around the inner pair.

```
a, b = ["x", "y"], [10, 20]
for i, (p, q) in enumerate(zip(a, b)):
    print(i, p, q)
d = {"k1": 1, "k2": 2}
for i, (k, v) in enumerate(d.items()):
    print(i, k, v)
```

OUTPUT

```
0 x 10
1 y 20
0 k1 1
1 k2 2
```

REMEMBER

- `enumerate(iterable, start=0)` yields `(index, item)` tuples.
- Unpack in the header: `for i, item in ...`; nested pairs need parentheses: `for i, (a, b) in ...`.
- Use `start=1` for human-friendly numbering: menus, rankings, line numbers.
- Prefer `enumerate` to `range(len(x))` in almost every case.
- Only need the items? Plain `for item in items` is simplest.
- It is lazy: use `list(enumerate(x))` when you want to print or reuse it.